FIELD NOTES

GitHub API Field Guide

The GitHub API stops working quietly: a list that returns thirty items, a webhook failing for a month, a 403 with no Retry-After. Every script here is read only.

88 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 88 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/github-api-fixes
Guides: allanninal.dev/github
All 14 repos: github.com/allanninal

The fixes

1 **a permission error is disguised as 404 Not Found**

The repository is open in a browser tab in front of you. The script asks for the same repository and gets 404 {"message":"Not Found"}. Somebody checks the spelling, then the owner name, then the case of both, then writes a ticket saying the API is broken. The API is not broken. It is refusing to tell you that the repository exists, because telling you would leak the existence of a private repository to a credential that has no business knowing about it.

`github_404_triage.py`

https://www.allanninal.dev/github/404-masking-403/

https://github.com/allanninal/github-api-fixes/tree/main/404-masking-403

---

2 **GITHUB_TOKEN gets 1,000 an hour, shared across the repo**

The script works on your laptop. It makes about 1,400 calls, finishes in two minutes, and nobody thinks about it again until it is moved into a workflow. There it gets to roughly call 1,000 and starts collecting 403. Same code, same repository, same endpoints. The only thing that changed is which credential is in the environment, and that credential was handed a ceiling a fifth the size of the one you tested against.

`github_actions_token_budget.py`

https://www.allanninal.dev/github/actions-token-repo-scoped-limit/

https://github.com/allanninal/github-api-fixes/tree/main/actions-token-repo-scoped-limit

---

3 **a hardcoded installation id stops matching reality**

The installation id was copied out of a settings URL in 2024 and pasted into an environment variable, where it has sat ever since. Last Tuesday an admin at one customer uninstalled the App during a security review and put it straight back, which they were entitled to do and which nobody told you about. Since then the token call for that customer has returned 404, and the error says nothing about installations.

`github_installation_id_drift.py`

https://www.allanninal.dev/github/app-installation-id-hardcoded/

https://github.com/allanninal/github-api-fixes/tree/main/app-installation-id-hardcoded

---

4 **a 404 that means the App is not installed on that repo**

The repository is public. You can open it in a browser without logging in. The App reads eleven other repositories in the same organization with the same token, and this one comes back 404 Not Found. So the search starts with permissions, and permissions are not the problem, because the App was never installed here at all.

`github_app_installation_presence.py`

https://www.allanninal.dev/github/app-not-installed-on-repo/

https://github.com/allanninal/github-api-fixes/tree/main/app-not-installed-on-repo

---

5 **the App was never subscribed to the event it waits for**

The handler was written, unit tested against a payload fixture, code reviewed and shipped. In production it has never run. Not once, not slowly, not with an error — the delivery log has thousands of entries and none of them is that event. There is nothing to debug, because nothing happened.

`github_app_event_subscriptions.py`

https://www.allanninal.dev/github/app-not-subscribed-to-event/

https://github.com/allanninal/github-api-fixes/tree/main/app-not-subscribed-to-event

---

6 **resource not accessible by integration on one endpoint**

Nineteen endpoints work. The twentieth returns 403 {"message":"Resource not accessible by integration"}, which names no permission, no resource and no level, and reads like a platform bug rather than a configuration one. It is the one failure in this cluster where GitHub actually tells you the answer: it is in the x-accepted-github-permissions response header on that very 403, which almost no HTTP client shows you by default.

`github_app_permission_diff.py`

https://www.allanninal.dev/github/app-permission-missing/

https://github.com/allanninal/github-api-fixes/tree/main/app-permission-missing

---

7 **a new App permission that installers never accepted**

The permission was added a fortnight ago. The App's settings page shows it, the pull request that used it was merged, and the feature works — for about two thirds of customers. The rest still get 403 {"message": "Resource not accessible by integration"} on exactly the call the permission was added for, and there is no pattern in which ones. Big orgs and small, old installs and new.

`github_permission_upgrade_lag.py`

https://www.allanninal.dev/github/app-permission-upgrade-not-accepted/

https://github.com/allanninal/github-api-fixes/tree/main/app-permission-upgrade-not-accepted

---

8 **the App's rate limit never grew with the installation**

The App serves an organization with four hundred repositories, and it throttles like a personal access token. Somebody quotes the number they remember — Apps get more, Apps scale with the installation, we will be fine at this volume — and the plan was built on it. GET /rate_limit says 5000, the same as the laptop script it replaced.

`github_app_limit_ceiling.py`

https://www.allanninal.dev/github/app-rate-limit-not-scaling/

https://github.com/allanninal/github-api-fixes/tree/main/app-rate-limit-not-scaling

---

9 **the installation token was narrowed below what the job needs**

The App is installed on the repository. You can see the installation, the repository is in its list, the permissions are generous, and one job gets 404 every time it touches that repository. The other four jobs, using the same App, on the same repository, are fine. Nobody has changed the App in months.

`github_token_reach.py`

https://www.allanninal.dev/github/app-token-scoped-down-too-far/

https://github.com/allanninal/github-api-fixes/tree/main/app-token-scoped-down-too-far

---

10 **the GitHub App has no webhook URL configured**

The App is installed on forty repositories, the permissions were approved, the event subscriptions are exactly right, and it has never once reacted to anything. There is nothing in the delivery log to read, no failures to group, no status codes to explain. Nothing is failing, because nothing is being attempted.

`github_app_hook_config.py`

https://www.allanninal.dev/github/app-webhook-url-unset/

https://github.com/allanninal/github-api-fixes/tree/main/app-webhook-url-unset

---

11 **401 Bad credentials on every endpoint, even public ones**

401 {"message":"Bad credentials"} on every endpoint you try, including ones that need no credential at all. The token is right there in the environment, it was working yesterday, and the error names nothing: not the account, not the scope, not which of the six things that produce this message actually happened.

`github_401_provenance.py`

https://www.allanninal.dev/github/bad-credentials-401/

https://github.com/allanninal/github-api-fixes/tree/main/bad-credentials-401

---

12 **the client still sends a username and password to the API**

401 {"message": "Support for password authentication was removed. Please use a personal access token instead."}. The reflex is to go and mint a token, which is right, and then to paste it where the password was, which is half right, and then to be surprised when a different call still fails. The credential was never the problem. The envelope was.

`github_auth_scheme_check.py`

https://www.allanninal.dev/github/basic-auth-password-removed/

https://github.com/allanninal/github-api-fixes/tree/main/basic-auth-password-removed

---

13 **branch protection is unreadable without admin, not absent**

The compliance report says nought out of two hundred and twelve repositories have branch protection. The security lead opens three of them at random and every one has protection on main, with required reviews and required checks, exactly as the policy says. The script is not lying about what it saw. It asked for the protection rules with a token that has read access and no more, GitHub answered 403 Must have admin rights to Repository., and somewhere in a try block that refusal became protected = False.

`github_branch_protection_audit.py`

https://www.allanninal.dev/github/branch-protection-requires-admin/

https://github.com/allanninal/github-api-fixes/tree/main/branch-protection-requires-admin

---

14 **a classic PAT passed its expiry and everything broke at once**

The integration ran for eleven months without anyone touching it. At 09:14 on a Tuesday every call started returning 401 Bad credentials, all at once, on every endpoint, with no deploy, no repository change and no org announcement. A classic personal access token reached its expiry date. So did a great many other theories that morning, and the API will not tell you which one was right.

`github_credential_differential.py`

https://www.allanninal.dev/github/classic-pat-expired/

https://github.com/allanninal/github-api-fixes/tree/main/classic-pat-expired

---

15 **code search is billed to its own 10 a minute bucket**

The script walks the org, calling GET /search/code once per repository. It gets through nine of them. The tenth returns 403, and GET /rate_limit says you have 4,987 requests left in the hour. Both statements are correct, because the request that was refused was never being counted in the bucket you just looked at.

`github_code_search_budget.py`

https://www.allanninal.dev/github/code-search-bucket-exhausted/

https://github.com/allanninal/github-api-fixes/tree/main/code-search-bucket-exhausted

---

16 **The scope is right, the account's role on the repo is read**

The bot has a token with the repo scope. It reads the repository, lists the pull requests, fetches the diff, and then the merge comes back 403. Somebody checks the token, sees repo ticked, mints a wider one with workflow and admin:org on it for good measure, and gets the same 403 back in the same millisecond. The scopes were never the ceiling. The account those scopes act on behalf of is a collaborator with read on that repository, and no token minted by that account will ever be able to merge anything into it.

`github_repo_role.py`

https://www.allanninal.dev/github/collaborator-permission-insufficient/

https://github.com/allanninal/github-api-fixes/tree/main/collaborator-permission-insufficient

---

17 **the compare endpoint stops at 250 commits and says nothing**

The release-notes generator diffs v4.2.0...v4.3.0 and produces a changelog that looks entirely plausible. It has 250 entries. The release contains 812 commits, and the ones it dropped are not the boring ones at the end — the shape of what came back is not what the code assumed at all.

`github_compare_truncation.py`

https://www.allanninal.dev/github/compare-250-commit-cap/

https://github.com/allanninal/github-api-fixes/tree/main/compare-250-commit-cap

---

18 **the deploy key is read-only and the push needs write**

The pipeline has cloned this repository twice a day for eighteen months. Somebody adds a step that pushes a version bump back, and it fails with ERROR: The key you are authenticating with has been marked as read only. — from Git, over SSH, with no HTTP status and nothing in the API logs. So the investigation starts in the wrong tool, goes through the known-hosts file and the agent and the private key in the secret store, and none of that is where the answer is. The answer is a boolean on an object the API will hand you in one request.

`github_deploy_key_capability.py`

https://www.allanninal.dev/github/deploy-key-read-only-assumed-write/

https://github.com/allanninal/github-api-fixes/tree/main/deploy-key-read-only-assumed-write

---

19 **the same webhook URL is registered on the org and the repo**

The bot comments twice on every pull request. Someone checks the receiver for a retry loop, finds none, and blames GitHub for sending duplicates. GitHub is sending exactly one copy of the event to each hook that asked for it — and two hooks asked, one on the repository and one on the organization, both pointing at the same URL.

`github_duplicate_hooks.py`

https://www.allanninal.dev/github/duplicate-webhooks/

https://github.com/allanninal/github-api-fixes/tree/main/duplicate-webhooks

---

20 **the endpoint ignores page and returns page one forever**

The collector increments a page counter, exactly as it does against a dozen other endpoints, and every request comes back 200 with a full page of rows. It never finishes. Either it runs until something kills it, or it stops at an arbitrary page cap and hands over a dataset that is the same thirty records repeated forty times. The endpoint has been returning page one to every request since the first one.

`github_page_param_ignored.py`

https://www.allanninal.dev/github/endpoint-ignores-page-param/

https://github.com/allanninal/github-api-fixes/tree/main/endpoint-ignores-page-param

---

21 **rotating the token invalidates every cached ETag at once**

The graph is a sawtooth and it is a very tidy one. Quota consumption sits near zero for fifty-odd minutes, jumps, and settles back, every hour, on the hour. Nothing in the schedule matches. The poller runs every thirty seconds and has done for months. What runs hourly is the installation token.

`github_etag_credential_check.py`

https://www.allanninal.dev/github/etag-invalidated-by-token-rotation/

https://github.com/allanninal/github-api-fixes/tree/main/etag-invalidated-by-token-rotation

---

22 **A 403 that means the feature is disabled, not the permission**

The security dashboard job gets 403 from /code-scanning/alerts. That is the status a missing grant produces, so somebody adds the security-events permission. Same 403. They swap the fine-grained token for an App installation with every security permission ticked. Same 403. Two weeks later a repository admin opens the settings page and code scanning has never been turned on for that repository, which is a checkbox, not a grant, and no credential in the world was going to open it.

`github_feature_flags.py`

https://www.allanninal.dev/github/feature-disabled-endpoint-403/

https://github.com/allanninal/github-api-fixes/tree/main/feature-disabled-endpoint-403

---

23 **Every call succeeds and reports on a fork, not the upstream**

The quarterly report says the platform repository had four merged pull requests, no releases and eleven open issues. It is a repository with nine thousand issues. Nothing failed to produce that report: every call returned 200, the pagination was followed properly, the retries never fired, the JSON parsed. The configuration was copied a year ago from an engineer's personal fork, and a fork is a different repository with its own issues, its own releases and its own branches. The integration has been right about the wrong object for four quarters.

`github_fork_or_upstream.py`

https://www.allanninal.dev/github/fork-vs-upstream-confusion/

https://github.com/allanninal/github-api-fixes/tree/main/fork-vs-upstream-confusion

---

24 **GraphQL returns 200 with an errors array and null data**

The call succeeded. response.ok is true, the status is 200, the body is valid JSON, and data.repository is null. Downstream something throws Cannot read property 'name' of null, or, far worse, nothing throws at all and the dashboard records that the repository has zero open pull requests. The reason is one line further down the body, in an errors array nobody wrote a branch for.

`github_graphql_envelope.py`

https://www.allanninal.dev/github/graphql-200-with-errors/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-200-with-errors

---

25 **Nobody measured what the GraphQL query actually costs**

The dashboard query has been running every fifteen seconds for a year. Last Tuesday somebody added one nested field to it, the review was two lines long and entirely reasonable, and on Thursday afternoon the whole integration started returning RATE_LIMITED at about ten past two. Nothing in the pull request said the price had gone from three points to fourteen, because nobody had ever written down that it was three.

`github_graphql_cost.py`

https://www.allanninal.dev/github/graphql-cost-not-measured/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-cost-not-measured

---

26 **A GraphQL connection asks for first: 500 and is rejected**

The query is fifteen lines long and it has never run once. The body comes back with Argument 'first' on Field 'issues' has an invalid value (500), there is no data key in it at all, and the 500 is not written anywhere in the document — it is the default on a variable somebody added a year ago. The code was ported from a REST client where per_page=500 quietly became 100 and the job kept working, so nobody ever learned that the number was wrong.

`github_graphql_slice.py`

https://www.allanninal.dev/github/graphql-first-over-100/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-first-over-100

---

27 **GraphQL node ids get stored where REST ids are expected**

The importer that reads through REST and the sync job that reads through GraphQL have been running side by side for a year. Neither has ever thrown. The join between their two tables returns nothing, so somebody widens it to a left join, sees a wall of nulls, and starts checking timestamps. There is nothing wrong with the timestamps. One table has 1347 in its id column and the other has MDU6SXNzdWUxMzQ3, and those are the same issue.

`github_graphql_id_crosswalk.py`

https://www.allanninal.dev/github/graphql-id-vs-databaseid/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-id-vs-databaseid

---

28 **A mutation costs five secondary points, a query costs one**

The read job runs all day without complaint. The write job, which sends fewer requests, dies eleven minutes in with 403 and You have exceeded a secondary rate limit. Somebody checks GET /rate_limit, sees four thousand GraphQL points still sitting there unspent, and concludes GitHub is wrong. GitHub is not wrong. The bucket that emptied is not the one being looked at, and the reason the write job reached it first is that every document containing a mutation is priced at five times a read.

`github_graphql_mutation_budget.py`

https://www.allanninal.dev/github/graphql-mutation-secondary-cost/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-mutation-secondary-cost

---

29 **Nested GraphQL connections truncate at 100 per parent**

The pagination loop is correct. It reads pageInfo.hasNextPage, follows endCursor, and walks every repository in the organisation without missing one. Inside each of those repositories the query also asks for pull requests, and every repository returns exactly one hundred of them, including the one that has four hundred and six. The response says so, in a totalCount sitting three lines above the truncated list, and nothing anywhere raises an error.

`github_graphql_nested.py`

https://www.allanninal.dev/github/graphql-nested-pagination-ignored/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-nested-pagination-ignored

---

30 **A nested GraphQL query requests more than 500,000 nodes**

The query worked all week against the test org, which has four repositories. Pointed at the real one it comes back rejected with MAX_NODE_LIMIT_EXCEEDED before a single row is read. The instinct is that the org is too big, so somebody reduces the date range and it fails identically, because the limit was never about how much data exists. It is about the numbers written in the query, and those did not change.

`github_graphql_nodes.py`

https://www.allanninal.dev/github/graphql-node-limit-exceeded/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-node-limit-exceeded

---

31 **GraphQL data is present but individual fields are null**

The query asked for fifty repositories and fifty came back. Nothing threw, nothing logged, the job took the usual eleven seconds. Eight of the fifty have null where diskUsage should be, because the token cannot read that field on a private repository, and the total that gets written to the dashboard is the sum of forty-two numbers presented as the sum of fifty. The response told you which eight, in an errors array that arrived alongside perfectly good data.

`github_graphql_partial.py`

https://www.allanninal.dev/github/graphql-partial-data-nulls/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-partial-data-nulls

---

32 **GraphQL points run out in a bucket separate from REST**

Every GraphQL call is coming back 200 with errors[0].type set to RATE_LIMITED. The health check is green, because the health check is a REST call and REST calls with the same token are working perfectly. Somebody is going to spend an hour looking for a difference between the two code paths, and the difference is not in the code: they are billed from two different buckets, and only one of them is empty.

`github_graphql_points.py`

https://www.allanninal.dev/github/graphql-rate-limited/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-rate-limited

---

33 **GraphQL search stops at the same 1,000 results as REST**

The REST search was hitting the thousand-result ceiling, loudly, with a 422 on page eleven that nobody could miss. So the query got rewritten in GraphQL, which is the modern API, has cursors instead of page numbers, and does not document a page limit anywhere obvious. The rewrite works. It runs to completion with no error at all, hasNextPage turns false, the loop exits cleanly, and it has collected 1,000 of the 18,231 issues that issueCount is reporting in the very same response.

`github_graphql_search_ceiling.py`

https://www.allanninal.dev/github/graphql-search-same-1000-cap/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-search-same-1000-cap

---

34 **A GraphQL query times out at 10s and is charged anyway**

The nightly job started failing with a 502 about ten seconds into a query that used to take four. The retry logic did the obvious thing and sent it again, three times, with backoff, exactly as it should for a gateway error. By the time somebody looked, the run had produced nothing at all and the hourly point budget was several hundred lighter than a run that produced nothing has any business being.

`github_graphql_timeout.py`

https://www.allanninal.dev/github/graphql-timeout-point-penalty/

https://github.com/allanninal/github-api-fixes/tree/main/graphql-timeout-point-penalty

---

35 **the installation covers only some repositories, silently**

The scanner reports clean. Every repository it looked at was fine, every check passed, and the summary at the bottom says so. It looked at twelve repositories. The organization has a hundred and forty. Nothing errored, nothing warned, and no line of that report is untrue — the App installation was set to selected repositories at some point in 2023 and the other hundred and twenty-eight have never been inside its field of view.

`github_app_coverage_audit.py`

https://www.allanninal.dev/github/installation-repository-selection-partial/

https://github.com/allanninal/github-api-fixes/tree/main/installation-repository-selection-partial

---

36 **the installation is suspended and every call it makes 403s**

Nothing was deployed and nothing was rotated. At some point on Thursday every call the App makes to one organization started coming back 403, and the webhook deliveries that used to arrive every few minutes stopped entirely. The App is still installed — you can see it in the list, the installation id you stored still resolves — and it does not work.

`github_installation_suspension.py`

https://www.allanninal.dev/github/installation-suspended/

https://github.com/allanninal/github-api-fixes/tree/main/installation-suspended

---

37 **the installation token expired an hour into the job**

The migration ran for fifty-eight minutes and processed eleven thousand repositories. Then every request started returning 401 Bad credentials — not some of them, all of them, from one second to the next. Restarting the job fixes it, which is the detail that sends everybody looking for a memory leak. It is not a leak. The process outlived its own credential.

`github_installation_token_age.py`

https://www.allanninal.dev/github/installation-token-expired/

https://github.com/allanninal/github-api-fixes/tree/main/installation-token-expired

---

38 **some endpoints refuse an installation token whatever it holds**

The App is installed. The JWT signs, the installation token mints, and nineteen calls in a row return exactly what they should. Then GET /user comes back 403 {"message": "Resource not accessible by integration"}, somebody adds a permission, every installer is emailed to accept the upgrade, and the same 403 arrives again the next morning.

`github_endpoint_audience.py`

https://www.allanninal.dev/github/installation-token-rejected-by-endpoint/

https://github.com/allanninal/github-api-fixes/tree/main/installation-token-rejected-by-endpoint

---

39 **clock drift puts the JWT iat claim in GitHub's future**

The same code, the same key, the same App. On a laptop it works every time. In the container it returns 401 {"message": "'Issued at' claim ('iat') must be an Integer representing the time that the assertion was issued"}, and on the third host it works again until Tuesday. Nothing about the JWT changed between those runs. What changed is which machine did the signing.

`github_clock_skew.py`

https://www.allanninal.dev/github/jwt-clock-drift-iat/

https://github.com/allanninal/github-api-fixes/tree/main/jwt-clock-drift-iat

---

40 **a GitHub App JWT that expires in an hour is refused**

The private key is right, the App exists, the signature verifies and the request still comes back 401 {"message": "'Expiration time' claim ('exp') is too far in the future"}. Three people regenerate the key. Somebody rewrites the signing code in a different library. The defect is one number, chosen three lines above the request, and it was wrong before anything was sent.

`github_app_jwt_claims.py`

https://www.allanninal.dev/github/jwt-exp-too-far-future/

https://github.com/allanninal/github-api-fixes/tree/main/jwt-exp-too-far-future

---

41 **the App JWT is signed with the wrong key or algorithm**

It worked until the key was rotated. Now every call is 401 {"message": "A JSON web token could not be decoded"}, or occasionally 404 {"message": "Integration not found"}, and the two arrive from what looks like the same deployment. The key is in the environment. The App exists. Somebody has already pasted the PEM into three different terminals to check it looks right.

`github_app_key_identity.py`

https://www.allanninal.dev/github/jwt-wrong-key-or-algorithm/

https://github.com/allanninal/github-api-fixes/tree/main/jwt-wrong-key-or-algorithm

---

42 **only the first page is read because the Link header is ignored**

The request returned 200. The JSON is a well-formed array of pull requests. Your audit read it, counted 30, and reported that the repository is tidy. There are 340 open pull requests. Nothing failed, nothing was logged, and the number you are now acting on is wrong by an order of magnitude — the answer was complete for one page and the rest was advertised in a header nobody read.

`github_link_header_audit.py`

https://www.allanninal.dev/github/link-header-not-followed/

https://github.com/allanninal/github-api-fixes/tree/main/link-header-not-followed

---

43 **the endpoint accepts a scope your token was never given**

Nineteen calls work. The twentieth returns 403 {"message": "Must have admin rights to Repository."}, or worse, a bare 404 on a repository that is open in a browser tab beside you. The token is valid, it is not expired, it has not been revoked, and it is missing exactly one word from the list it was created with.

`github_scope_diff.py`

https://www.allanninal.dev/github/missing-oauth-scope/

https://github.com/allanninal/github-api-fixes/tree/main/missing-oauth-scope

---

44 **polling without ETags spends full quota on unchanged data**

The dashboard polls eight endpoints every thirty seconds and burns through 5,000 requests before lunch. Almost nothing it fetches has changed. GitHub has been sending an etag on every one of those responses, and every response that comes back 304 Not Modified is free — it does not count against the rate limit at all. This is the one problem in this section where the fix pays for itself in a number you can print.

`github_etag_saving.py`

https://www.allanninal.dev/github/no-conditional-requests/

https://github.com/allanninal/github-api-fixes/tree/main/no-conditional-requests

---

45 **one user revoked your app and only their token is dead**

One customer's sync has been failing since Thursday. Everybody else is fine. The token has not expired, because OAuth user tokens do not expire; nothing was deployed; the error is 401 Bad credentials and the retry loop has been dutifully re-attempting it every fifteen minutes for four days. The user clicked Revoke on a settings page you will never see.

`github_user_token_liveness.py`

https://www.allanninal.dev/github/oauth-token-revoked-by-user/

https://github.com/allanninal/github-api-fixes/tree/main/oauth-token-revoked-by-user

---

46 **a read-only job holds a token that can delete repositories**

There is no incident to attach this to. The job runs every ten minutes, lists open pull requests, posts a count to a dashboard and has not failed once this year. The token it uses was created in four seconds by ticking repo, and it can force-push to every private repository in the organization, rewrite webhooks and delete the lot.

`github_scope_blast_radius.py`

https://www.allanninal.dev/github/over-scoped-token/

https://github.com/allanninal/github-api-fixes/tree/main/over-scoped-token

---

47 **per_page is unset so every list costs 3.3x more requests**

The job is correct. It follows rel="next" to the end, it reads every issue, and it burns through the hourly quota by lunchtime. Nothing is broken and nothing needs debugging — it is simply making three and a third times as many requests as it needs to, because per_page was never set and the default is 30.

`github_per_page_audit.py`

https://www.allanninal.dev/github/per-page-default-30/

https://github.com/allanninal/github-api-fixes/tree/main/per-page-default-30

---

48 **per_page above 100 is clamped and never rejected**

Somebody set per_page=500 to cut the number of round trips, and the request came back with exactly one hundred items and a 200. One hundred is fewer than five hundred, so the loop concluded it had reached the end and stopped. There was no 422, no warning header and nothing in the logs. Four fifths of the collection was never read, and the report built on it looks entirely finished.

`github_per_page_clamp.py`

https://www.allanninal.dev/github/per-page-over-100-clamped/

https://github.com/allanninal/github-api-fixes/tree/main/per-page-over-100-clamped

---

49 **the x-poll-interval header is ignored on events endpoints**

The events consumer polls every five seconds because five seconds felt responsive. It gets the same page back seven hundred times an hour. On every one of those responses GitHub has been returning a header that says how long to wait before the next one, and the client has never read it.

`github_poll_interval_check.py`

https://www.allanninal.dev/github/poll-interval-header-ignored/

https://github.com/allanninal/github-api-fixes/tree/main/poll-interval-header-ignored

---

50 **the integration polls for events a webhook would push**

The integration works. It notices new pull requests, it picks up comments, it has never lost anything. It also makes 4,320 requests an hour to do it, notices each of those events an average of thirty seconds after it happened, and would notice nothing at all if the poll ran once a day instead. There is no bug here. There is a design that was never revisited.

`github_webhook_vs_poll.py`

https://www.allanninal.dev/github/polling-instead-of-webhooks/

https://github.com/allanninal/github-api-fixes/tree/main/polling-instead-of-webhooks

---

51 **a pull request's files and commits lists are both capped**

The review bot posts its summary on a nine-hundred-file pull request and says three files changed. Nobody reads past the first line, because a bot that has been right for a year is not a thing people audit. The pull request object knew the real number the whole time. It is just that changed_files lives on the pull request and the file list lives one URL further down, and nothing in either response mentions the other.

`github_pr_truncation.py`

https://www.allanninal.dev/github/pr-files-and-commits-caps/

https://github.com/allanninal/github-api-fixes/tree/main/pr-files-and-commits-caps

---

52 **A repository went private and anonymous callers now see 404**

The dependency dashboard has read one repository's tags and release notes since 2019, anonymously, because it is a public repository and there was never any reason to authenticate. On a Tuesday it starts returning 404. Nothing was deployed, no code changed, the URL is character-for-character the one that worked on Monday. Somebody checks whether the repository was deleted, finds it in their browser because their browser is logged in, and now has two contradictory facts and no idea which to believe.

`github_visibility_change.py`

https://www.allanninal.dev/github/private-repo-visibility-changed/

https://github.com/allanninal/github-api-fixes/tree/main/private-repo-visibility-changed

---

53 **core REST quota is exhausted and every call returns 403**

Every endpoint fails at once, with the same status, in the same second. That pattern says outage or bad credentials, and it is neither: the hourly bucket is empty, and an empty bucket refuses everything equally. The awkward part is that by the time you are reading the 403 the interesting question has already gone past. Not are we out, but at what rate did we spend it, and would that rate have fitted.

`github_quota_forecast.py`

https://www.allanninal.dev/github/rate-limit-core-exhausted/

https://github.com/allanninal/github-api-fixes/tree/main/rate-limit-core-exhausted

---

54 **requests go out anonymous and are capped at 60 an hour**

It works on the first run and dies on the fourth. On a laptop it survives a couple of minutes; in CI, behind a shared NAT address, it is refused almost immediately and the run before yours gets the blame. Nothing in the code is wrong. The token simply is not arriving, and GitHub does not treat that as an error. It serves you anyway, as a stranger, from a bucket sixty deep.

`github_auth_tier_check.py`

https://www.allanninal.dev/github/rate-limit-unauthenticated/

https://github.com/allanninal/github-api-fixes/tree/main/rate-limit-unauthenticated

---

55 **the Link header has no rel=last so the page count breaks**

The pager was written properly. It reads the Link header, it pulls the page number out of rel="last", and it uses that number to size the job before it starts: a progress bar, a worker pool over the page range, an estimate in the log line. On one endpoint the header comes back with a rel="next" and no rel="last" at all, and the job reports that it read one page of a one-page collection. It read one page of nine hundred.

`github_rel_last_absent.py`

https://www.allanninal.dev/github/rel-last-absent/

https://github.com/allanninal/github-api-fixes/tree/main/rel-last-absent

---

56 **the repository is archived so every write returns 403**

The labelling bot has been failing on one repository since March. Everything it reads comes back perfectly: issues, labels, the repository object itself, all 200. The moment it tries to add a label it gets 403, so the on-call runbook says permissions, so somebody widens the token, and it still gets 403. The token was never the problem. Somebody archived the repository seven months ago, which makes it read-only for everyone and every credential, and the bot has been retrying a request that will not be accepted by anyone until it is unarchived.

`github_archived_repo_guard.py`

https://www.allanninal.dev/github/repo-archived-writes-403/

https://github.com/allanninal/github-api-fixes/tree/main/repo-archived-writes-403

---

57 **the repository is disabled and behaves like a ghost**

The weekly platform report says the organisation has one repository with no branch protection, no webhooks, no open pull requests and no contributors. It is not a new repository and it is not empty; it shipped for three years. Every call the report makes against it either 404s or comes back with nothing in it, while the repository itself reads fine and sits in the organisation listing next to everything else. One boolean on the repository object explains all of it, and it is not archived.

`github_disabled_repo_probe.py`

https://www.allanninal.dev/github/repo-disabled/

https://github.com/allanninal/github-api-fixes/tree/main/repo-disabled

---

58 **the repository was renamed and every call now 301s**

Somebody renamed the repository on a Tuesday. Nothing broke, which is the problem: GitHub left a redirect at the old path and most clients follow it without mentioning it, so the integration kept working against a name that no longer exists. The config still says acme/platform-api. The repository is called acme/core-api. Every request the job makes takes two round trips instead of one, and has done for eight months.

`github_repo_renamed.py`

https://www.allanninal.dev/github/repo-renamed-301-redirect/

https://github.com/allanninal/github-api-fixes/tree/main/repo-renamed-301-redirect

---

59 **expensive requests are killed at ten seconds with a 502**

One call in the whole integration returns 502 Bad Gateway. Everything else is fine, the token is fine, the status page is green, and the call works on every repository except the big one. So the retry wrapper does what retry wrappers do: it waits a second and issues the identical expensive request, which takes the identical ten seconds and fails in the identical way, three times, before the job gives up and pages somebody.

`github_timeout_502.py`

https://www.allanninal.dev/github/request-timeout-502/

https://github.com/allanninal/github-api-fixes/tree/main/request-timeout-502

---

60 **403 Resource not accessible by personal access token**

The token was minted this morning with the permissions somebody carefully ticked, it reads the repository perfectly well, and one call comes back 403 {"message": "Resource not accessible by personal access token"}. The obvious next move is to look at what the token holds and compare it against what the endpoint wanted. Half of that is possible. The endpoint will tell you what it wanted. Nothing anywhere will tell you what the token holds.

`github_fine_grained_pat_probe.py`

https://www.allanninal.dev/github/resource-not-accessible-by-pat/

https://github.com/allanninal/github-api-fixes/tree/main/resource-not-accessible-by-pat

---

61 **the client ignores retry-after and keeps hammering the API**

The log shows four hundred consecutive 403s, one second apart, for eleven minutes. The retry logic is working perfectly: it catches the error, waits, tries again, never gives up. GitHub said retry-after: 120 on the very first one, and the client threw that header away along with the rest of the response.

`github_backoff_plan.py`

https://www.allanninal.dev/github/retry-after-ignored/

https://github.com/allanninal/github-api-fixes/tree/main/retry-after-ignored

---

62 **org lists silently omit SSO-enforced organizations**

The user belongs to six organizations. GET /user/orgs returns four. Status 200, valid JSON, no errors array, nothing in the body that hints anything is absent. The two organizations that enforce SAML single sign-on for a token that was never authorized against them are simply not in the list, and the only place GitHub mentions it is a response header called X-GitHub-SSO.

`github_sso_partial_results.py`

https://www.allanninal.dev/github/saml-partial-results/

https://github.com/allanninal/github-api-fixes/tree/main/saml-partial-results

---

63 **search returns at most 1,000 results whatever total_count says**

The response says "total_count": 24831. You page through it at 100 per request, and on page 11 the API returns 422 Validation Failed with "Only the first 1000 search results are available". The count was true. The results were never there to fetch — total_count describes the match set, not the part of it you are allowed to page through.

`github_search_cap_audit.py`

https://www.allanninal.dev/github/search-1000-result-cap/

https://github.com/allanninal/github-api-fixes/tree/main/search-1000-result-cap

---

64 **search has its own 30-per-minute bucket and drains separately**

The job loops over four hundred repositories and runs one search in each. Around the thirtieth it starts returning 403, and the part that makes no sense is that everything else keeps working: the repository reads in the same loop, on the same token, in the same second, are fine. Two buckets, and only one of them is empty. The error message does not mention which.

`github_search_budget.py`

https://www.allanninal.dev/github/search-bucket-exhausted/

https://github.com/allanninal/github-api-fixes/tree/main/search-bucket-exhausted

---

65 **incomplete_results is true and the search answer is partial**

The same search runs every morning and the number it reports moves: 412 on Monday, 380 on Tuesday, 412 again on Wednesday. Nothing failed. Every response was a 200 with well-formed JSON and a sensible-looking list of items. The field that says the answer was cut short is sitting at the top of every one of those payloads, and no code in the pipeline has ever read it.

`github_search_incomplete.py`

https://www.allanninal.dev/github/search-incomplete-results/

https://github.com/allanninal/github-api-fixes/tree/main/search-incomplete-results

---

66 **over 100 concurrent requests trips a secondary rate limit**

Someone replaces a for loop with a Promise.all and the job goes from nine minutes to forty seconds, once. The next run returns 403 on two thirds of the requests. You check the quota, because a 403 from GitHub means the quota, and the quota says four thousand eight hundred requests left. Both numbers are true. They are about different limits.

`github_concurrency_probe.py`

https://www.allanninal.dev/github/secondary-limit-concurrency/

https://github.com/allanninal/github-api-fixes/tree/main/secondary-limit-concurrency

---

67 **bulk issue or comment creation exceeds 80 requests a minute**

The migration script imports 2,400 issues from the old tracker. It gets through about eighty of them, then every remaining call comes back 403. You check the quota and there are 4,900 requests left in it. The limit that stopped you is not counting requests. It is counting the things you created.

`github_content_burst_audit.py`

https://www.allanninal.dev/github/secondary-limit-content-creation/

https://github.com/allanninal/github-api-fixes/tree/main/secondary-limit-content-creation

---

68 **a hot endpoint burns 900 points a minute and gets throttled**

One endpoint in the job keeps failing and the rest are fine. It fails for about a minute, recovers, and fails again twenty minutes later. The hourly quota is barely touched and the concurrency is one, because someone already serialised it after the last incident. What is left is the cap nobody budgets for: not how many requests you make, but how much work you asked one path to do inside sixty seconds.

`github_endpoint_cost_audit.py`

https://www.allanninal.dev/github/secondary-limit-points-per-minute/

https://github.com/allanninal/github-api-fixes/tree/main/secondary-limit-points-per-minute

---

69 **the token expires in days and nothing is watching the clock**

Nothing is broken. That is the entire point of this note: it is the one that runs before the outage rather than after it. Somewhere in your environment is a credential with six days left on it, and the only thing standing between you and a 09:14 Tuesday is that nobody has read a header GitHub has been sending on every single response for months.

`github_token_expiry_watch.py`

https://www.allanninal.dev/github/token-expiring-soon/

https://github.com/allanninal/github-api-fixes/tree/main/token-expiring-soon

---

70 **the token is passed as an access_token query parameter**

The call returns 401 {"message": "Requires authentication"} for an endpoint that plainly ought to work, and the token in the URL is correct — you can paste it into a header and watch the same call succeed. That is the small half of this. The large half is that the URL is now in an access log, a CI transcript, a browser history and a support ticket, and nothing about that half produces an error at all.

`github_token_in_url.py`

https://www.allanninal.dev/github/token-in-query-string/

https://github.com/allanninal/github-api-fixes/tree/main/token-in-query-string

---

71 **rows move between pages and the walk skips records**

The sync reads the Link header, follows rel="next" to the very end and asks for a hundred items a page. It is, by every check in this section, a correct pagination loop. It also misses about one issue a night and occasionally imports the same one twice, and nobody can reproduce it, because reproducing it requires somebody to touch an issue during the eleven seconds the walk is passing over it.

`github_unstable_sort.py`

https://www.allanninal.dev/github/unstable-sort-duplicates/

https://github.com/allanninal/github-api-fixes/tree/main/unstable-sort-duplicates

---

72 **a pinned X-GitHub-Api-Version stopped being supported**

Nothing was deployed. No token was rotated, no permission changed, no dependency was upgraded. On a Tuesday morning every request to the GitHub API starts coming back refused, with a message about an API version, and the only thing that moved is the date. The header responsible was copied out of a documentation sample in 2022 and has not been looked at since.

`github_api_version_pin.py`

https://www.allanninal.dev/github/unsupported-api-version/

https://github.com/allanninal/github-api-fixes/tree/main/unsupported-api-version

---

73 **a classic token nobody used for a year is deleted for you**

The restore drill is on a Tuesday morning and the token that has been sitting in the vault since the runbook was written answers 401. It has not expired, because it was created with no expiry. Nobody revoked it. It is not in the account's token list at all, because GitHub removed it for going a year without being used, and the reason it went a year without being used is that it is a token for emergencies.

`github_token_dormancy.py`

https://www.allanninal.dev/github/unused-classic-token-auto-revoked/

https://github.com/allanninal/github-api-fixes/tree/main/unused-classic-token-auto-revoked

---

74 **every request 403s because there is no User-Agent header**

403 {"message":"Request forbidden by administrative rules. Please make sure your request has a User-Agent header."} — and it happens on the REST root, which any anonymous caller can read. Nobody looks at the body, because 403 in this API almost always means quota or permissions, and both of those theories send you somewhere the answer is not.

`github_user_agent_403.py`

https://www.allanninal.dev/github/user-agent-missing/

https://github.com/allanninal/github-api-fixes/tree/main/user-agent-missing

---

75 **the hook sends form-encoded bodies to a JSON receiver**

The handler is written, deployed and subscribed to the right event. GitHub says the delivery succeeded. Your receiver says it got a request and found nothing in it — or says nothing at all, because it returned 200 having parsed an empty object out of a body it never understood. Every field in the hook looks right, because the field that is wrong is the one nobody reads.

`github_hook_content_type.py`

https://www.allanninal.dev/github/webhook-content-type-mismatch/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-content-type-mismatch

---

76 **webhook deliveries are failing and nobody reads the log**

Someone opens a pull request and the bot that should comment on it says nothing. You grep your receiver's logs for the last hour and find no request at all, which points the finger at GitHub. It is not GitHub. The delivery happened, your server answered 502, and GitHub wrote that down in a log you have never opened.

`github_hook_delivery_audit.py`

https://www.allanninal.dev/github/webhook-deliveries-failing/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-deliveries-failing

---

77 **the hook is not subscribed to the event you are waiting for**

The handler was written, reviewed, unit tested against a saved payload, and deployed. In production it has never executed once. There is no error anywhere because there is no failure anywhere: the hook was created years ago with push and pull_request, and the event your handler waits for has never been sent to it.

`github_hook_event_coverage.py`

https://www.allanninal.dev/github/webhook-event-not-subscribed/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-event-not-subscribed

---

78 **the webhook posts your payloads to an http:// URL**

The hook works. It has a secret, it is subscribed to the right events, the delivery log is a column of 200s, and insecure_ssl reads 0, which is the field the last security review looked at. The URL is http://hooks.internal.acme.io/github, so every payload and the signature that authenticates it have been crossing the network as plain text since the day it was created.

`github_hook_transport.py`

https://www.allanninal.dev/github/webhook-http-url/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-http-url

---

79 **the webhook exists but somebody switched it off**

The hook is right there in the repository's settings, pointed at the right URL, subscribed to the right events, with a secret set and a green tick beside it from the last time anyone looked. The delivery log is not full of failures. It is empty, and it has been empty for five weeks. Nothing is broken because nothing is running: active is false.

`github_hook_active_audit.py`

https://www.allanninal.dev/github/webhook-inactive/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-inactive

---

80 **SSL verification is switched off on the webhook**

The URL starts with https. The delivery log is clean, every attempt is a 200, and the hook has a secret so the payloads are signed. Everything a review asks about is green. One field in the same config object says "insecure_ssl": "1", which means GitHub has not looked at your certificate since somebody set that during the initial deploy in 2023.

`github_hook_ssl_verification.py`

https://www.allanninal.dev/github/webhook-insecure-ssl/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-insecure-ssl

---

81 **a firewall allow-list no longer matches GitHub's hook IPs**

Some deliveries land and some do not, with no pattern anybody can find. It is not the event type, it is not the payload size, it is not the time of day. The hook is configured correctly, the receiver is up, and the delivery log shows connection failures scattered among successes. Nothing is wrong with the webhook. The problem is a text file on a firewall, written correctly two years ago.

`github_meta_hook_ranges.py`

https://www.allanninal.dev/github/webhook-ip-allowlist-drift/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-ip-allowlist-drift

---

82 **a webhook with no secret sends no signature to verify**

The receiver has a signature check. It reads X-Hub-Signature-256, computes an HMAC over the body, compares in constant time, and returns 401 when they differ. It has never returned 401, because the hook it serves has no secret, so GitHub does not send the header, and the check quietly skips itself on every request.

`github_hook_secret_audit.py`

https://www.allanninal.dev/github/webhook-no-secret/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-no-secret

---

83 **the webhook secret is set and has never been rotated**

There is no symptom. The hook works, the signatures verify, the deliveries are green, and the secret that authenticates every event has been sitting in the same configuration file since the integration was built. It has outlasted two contractors, a laptop that was never wiped, and a support ticket where somebody pasted the receiver's environment into a chat window to get help.

`github_hook_secret_age.py`

https://www.allanninal.dev/github/webhook-secret-never-rotated/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-secret-never-rotated

---

84 **the receiver still checks the legacy SHA-1 signature**

The receiver verifies. It reads a signature header, computes an HMAC over the raw body, compares in constant time and returns 401 when they differ. It has been doing that since 2017, which is the problem: the header it reads is X-Hub-Signature, the SHA-1 one GitHub keeps sending for the sake of receivers exactly like this one.

`github_hook_signature_headers.py`

https://www.allanninal.dev/github/webhook-sha1-signature-only/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-sha1-signature-only

---

85 **the receiver takes longer than 10 seconds and times out**

The delivery log has started showing timed out against a handler that, according to your own logs, finished the job perfectly. It ran, it did the work, it wrote the record, it returned 200 — twelve seconds after it started. GitHub stopped listening at ten and filed the delivery as a failure, and the redelivery it may send will do the same twelve seconds of work again.

`github_hook_delivery_duration.py`

https://www.allanninal.dev/github/webhook-timeout-10s/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-timeout-10s

---

86 **the hook subscribes to every event with a wildcard**

The hook was set up in a hurry, with * in the events list, because nobody was sure yet which events the integration would need. That was two years ago. The receiver now handles four event types and is delivered somewhere north of forty, and every one of the ones it does not want still arrives, still gets its signature verified, and is still thrown away. Nothing has ever failed.

`github_hook_event_volume.py`

https://www.allanninal.dev/github/webhook-wildcard-events/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-wildcard-events

---

87 **a JWT sent as token, and the 401 blames the credential**

The App is registered, the private key is the right one, the JWT is freshly signed and it verifies in three different debuggers. GitHub answers 401 {"message":"Bad credentials"}. The credential in that request is perfect. The word in front of it is not, and the message never mentions words.

`github_auth_scheme.py`

https://www.allanninal.dev/github/wrong-authorization-scheme/

https://github.com/allanninal/github-api-fixes/tree/main/wrong-authorization-scheme

---

88 **the automation runs as a person who can leave the company**

The release notes are signed by someone who left in March. Every automated review comment carries a colleague's avatar, every bot commit says their name, and none of it is a display problem: the token doing the work was minted on their account four years ago, so the integration is not running as a service. It is running as them.

`github_actor_identity.py`

https://www.allanninal.dev/github/wrong-identity-token/

https://github.com/allanninal/github-api-fixes/tree/main/wrong-identity-token

---